concern is an obvious one for with decentralized public participation, in all these cases, you must be aware that data transmitted through the function parameters--such as bid values in a blind auction--has to be kept private and secure.

We'll examine in section 5.4 how a combination of cryptography and hashing is used to address these issues. Thus cryptography is used not only in establishing the identity of a decentralized participant, but also in secure hashing for privacy and security.

5.3 Hashing basics

Hashing is a transformation that maps data of arbitrary sizes to a standard fixed-size value. The *hash* of data elements are computed by using a hash function, as shown here:

```
hash = hashFunction(one or more data items)
```

Using a logical XOR (exclusive OR) function as a simple hash function and two data items of a =1010 binary, b= 1100 binary, you get the hashed value of the two data items as 0110:

```
hash value = xor(a=1010, b=1100) = 0110
```

> **DEFINITION** *Hashing* is the process of mapping data of an arbitrary length to a fixed size by using a specially defined function called a *hash function*.

Even a single bit change in the data elements changes the hash value of the data elements significantly. Any type of data, including a database or an image, can be succinctly represented by a hash of fixed length, as shown in figure 5.9. A 256-bit data item and a strong hash function together provide a large, collision-free account address space. *Collision-free* means there is a high probability that no two values generated by the hash function will be the same and that you'll get a unique hash value when you apply the hashing function to the same data elements. This property is an important requirement of a hash